Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційні систем та технологій

Лабораторна робота № 4

із дисципліни «Технології розроблення програмного забезпечення»

Тема: «Вступ до паттернів проектування»

Виконав
Студент групи ІА-31:
Козир Я. О.

Перевірив:
Мягкий М. Ю.

Київ 2025

Зміст

1. Мета: 3
2. Хід роботи: 3
   1) Ознайомитись з короткими теоретичними відомостями. 3
   2) Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей. 3
   3) Реалізувати один з розглянутих шаблонів за обраною темою. 6
   4) Реалізувати не менше 3-х класів відповідно до обраної теми. 7
   5) Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону. 12
3. Контрольні питання: 12
4. Висновок 15

1. **Мета:**

Вивчити структуру шаблонів «Singleton», «Iterator», «Proxy», «State», «Strategy» та навчитися застосовувати їх в реалізації програмної системи.

2. **Хід роботи:**

Тема : Online radio station

1) Ознайомитись з короткими теоретичними відомостями.

Singleton: Забезпечує єдиний екземпляр класу з глобальним доступом. Використовується для унікальних ресурсів (наприклад, конфігураційні файли), але вважається антипатерном через глобальний стан.
Iterator: Дозволяє послідовний доступ до елементів колекції без розкриття її внутрішньої структури. Підтримує різні методи обходу (наприклад, у глибину, випадково).
Proxy: Діє як замінник або заглушка для іншого об’єкта, додаючи функціонал (наприклад, ледаче завантаження, контроль доступу). Зменшує кількість запитів до зовнішніх сервісів (наприклад, оптимізація DocuSign).
State: Дозволяє змінювати поведінку об’єкта залежно від його стану (наприклад, типи карток або режими системи). Використовує окремі класи для кожного стану.
Strategy: Уможливлює заміну алгоритмів поведінки об’єкта (наприклад, методи сортування чи маршрути). Відокремлює логіку алгоритмів від контексту для гнучкості.

2) Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.

Реалізовано функціонал керування станами DJ-стріму (DjStream) за допомогою патерну "Стан". Це дозволяє стріму змінювати свою поведінку (реагувати на команди "Старт", "Стоп", "Пауза", "Відновити") залежно від того, чи він зараз зупинений, відтворюється чи на паузі. Взаємодія відбувається через клас DjStream (контекст), який делегує виклики методів поточному об'єкту стану (IStreamState).

DjStream.cs M

adioStation.Domain > DjStream.cs > DjStream

```
1   using System;
2   using System.ComponentModel.DataAnnotations;
3
4   namespace OnlineRadioStation.Domain
5   {
        20 references
6       public class DjStream
7       {
            6 references
8           private IStreamState _currentState;
9           [Key]
            2 references
10          public Guid StreamId { get; set; }
            1 reference
11          public Guid StationId { get; set; }
            2 references
12          public Guid DjId { get; set; }
            1 reference
13          public DateTime StartTime { get; set; }
            2 references
14          public DateTime? EndTime { get; set; }
15
            0 references
16          public RadioStationEntity Station { get; set; } = null!;
            0 references
17          public User Dj { get; set; } = null!;
18
            1 reference
19          public DjStream()
20          {
21              _currentState = new StoppedState(this);
22          }
23
            5 references
24          public void SetState(IStreamState newState)
25          {
26              _currentState = newState;
27              Console.WriteLine($"Stream state changed to: {newState.GetType().Name}");
28          }
29
            1 reference
30          public void StartStream()
31          {
32              _currentState.Play();
33          }
34
            1 reference
35          public void StopStream()
36          {
37              _currentState.Stop();
38          }
39
            1 reference
40          public void PauseStream()
41          {
42              _currentState.Pause();
43          }
44
            1 reference
45          public void ResumeStream()
46          {
47              _currentState.Resume();
48          }
49      }
50  }
```

DjStream (Контекст, що використовує патерн Стан)

DjStream.cs M ● PlayingState.cs U ●

RadioStation.Domain > PlayingState.cs > PlayingState > PlayingState

```
using System;
namespace OnlineRadioStation.Domain
{
    3 references
    public class PlayingState : IStreamState
    {
        5 references
        private readonly DjStream _stream;

        2 references
        public PlayingState(DjStream stream)
        {
            _stream = stream;
        }
        2 references
        public void Play()
        {
            Console.WriteLine("Stream is already playing.");
        }
        2 references
        public void Stop()
        {
            Console.WriteLine("Stream is stopping...");
            _stream.SetState(new StoppedState(_stream));
        }
        2 references
        public void Pause()
        {
            Console.WriteLine("Stream is pausing...");
            _stream.SetState(new PausedState(_stream));
        }
        2 references
        public void Resume()
        {
            Console.WriteLine("Stream is already playing, cannot resume.");
        }
    }
}
```

PlayingState.cs (Конкретний стан)

```
DjStream.cs M ●   PlayingState.cs U ●   IStreamState.cs
RadioStation.Domain > IStreamState.cs > ...
1   using System.Threading.Tasks;
2
3   namespace OnlineRadioStation.Domain
4   {
        5 references
5       public interface IStreamState
6       {
            4 references
7           void Play();
8
            4 references
9           void Stop();
10
            4 references
11          void Pause();
12
            4 references
13          void Resume();
14      }
15  }
```

IstreamState.cs (інтерфейс), що визначає контракт для станів

3) Реалізувати один з розглянутих шаблонів за обраною темою.

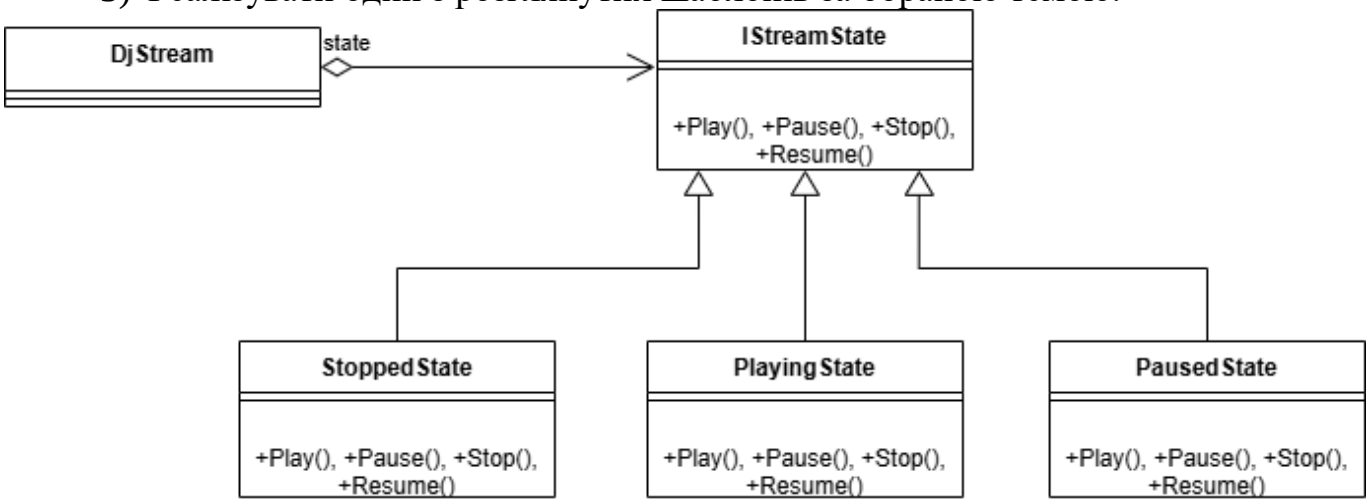


Рисунок 1 - Структура патерну Стан (State)

Патерн **Стан (State)** для DjStream
Призначення патерну: Дозволяє об'єкту змінювати свою поведінку при зміні внутрішнього стану. Здається, що об'єкт змінює свій клас.

Як застосуємо у проєкті: Ми використаємо цей патерн для керування станами об'єкта DjStream (наприклад: Зупинено, Відтворюється, На паузі). Залежно від поточного стану, дії (як-от "Почати відтворення", "Зупинити") будуть виконуватися по-різному.

IStreamState (Інтерфейс Стану) - інтерфейс, який описує всі можливі дії, що можна виконати зі стрімом: Play(), Stop(), Pause(), Resume(). Він визначає "контракт" для всіх конкретних станів.

StoppedState, PlayingState, PausedState (Конкретні Стани) - класи, що реалізують інтерфейс IStreamState. Кожен клас відповідає за поведінку стріму в конкретному стані.

- У StoppedState метод Play() переведе стрім у PlayingState, а метод Stop() нічого не робитиме.
- У PlayingState метод Stop() переведе стрім у StoppedState, Pause() — у PausedState, а Play() нічого не робитиме.
- У PausedState метод Resume() переведе стрім у PlayingState, а Stop() — у StoppedState.

DjStream (Контекст) - клас, стан якого ми керуємо. Він буде містити посилання на поточний об'єкт стану (IStreamState currentState).Методи класу DjStream (наприклад, StartStream(), StopStream(), PauseStream()) будуть просто викликати відповідні методи поточного стану (currentState.Play(), currentState.Stop(), currentState.Pause()). Ці ж методи DjStream відповідатимуть за зміну поточного стану (наприклад, currentState = new PlayingState(this);).

4) Реалізувати не менше 3-х класів відповідно до обраної теми.

Для реалізації патерну "Стан" було створено наступні типи, що відповідають вимозі створення щонайменше 3-х класів: інтерфейс IStreamState, що визначає загальний контракт станів, та класи StoppedState, PlayingState, PausedState, що реалізують поведінку для кожного конкретного стану DjStream.

DjStream.cs M ● PlayingState.cs U ● IStreamState.cs

RadioStation.Domain > IStreamState.cs > ...

```
1   using System.Threading.Tasks;
2
3   namespace OnlineRadioStation.Domain
4   {
        5 references
5       public interface IStreamState
6       {
            4 references
7           void Play();
8
            4 references
9           void Stop();
10
            4 references
11          void Pause();
12
            4 references
13          void Resume();
14      }
15  }
```

IstreamState.cs(визначення інтерфейсу з усіма методами (Play, Stop, Pause, Resume).

DjStream.cs M ● | PlayingState.cs U ● | IStreamState.cs U ● | StoppedState.cs U ●

RadioStation.Domain > StoppedState.cs > ...

```
1   using System;
2   namespace OnlineRadioStation.Domain
3   {
        4 references
4       public class StoppedState : IStreamState
5       {
            3 references
6           private readonly DjStream _stream;
            3 references
7           public StoppedState(DjStream stream)
8           {
9               _stream = stream;
10          }
            2 references
11          public void Play()
12          {
13              Console.WriteLine("Stream is starting...");
14              _stream.SetState(new PlayingState(_stream));
15          }
            2 references
16          public void Stop()
17          {
18              Console.WriteLine("Stream is already stopped.");
19          }
            2 references
20          public void Pause()
21          {
22              Console.WriteLine("Cannot pause a stopped stream.");
23          }
24
            2 references
25          public void Resume()
26          {
27              Console.WriteLine("Cannot resume a stopped stream.");
28          }
29      }
30  }
```

Клас StoppedState, що реалізує IstreamState

DjStream.cs M ● PlayingState.cs U ●

RadioStation.Domain > PlayingState.cs > PlayingState > PlayingState

```
1   using System;
2   namespace OnlineRadioStation.Domain
3   {
        3 references
4       public class PlayingState : IStreamState
5       {
            5 references
6           private readonly DjStream _stream;
7
            2 references
8           public PlayingState(DjStream stream)
9           {
10              _stream = stream;
11          }
            2 references
12          public void Play()
13          {
14              Console.WriteLine("Stream is already playing.");
15          }
            2 references
16          public void Stop()
17          {
18              Console.WriteLine("Stream is stopping...");
19              _stream.SetState(new StoppedState(_stream));
20          }
            2 references
21          public void Pause()
22          {
23              Console.WriteLine("Stream is pausing...");
24              _stream.SetState(new PausedState(_stream));
25          }
            2 references
26          public void Resume()
27          {
28              Console.WriteLine("Stream is already playing, cannot resume.");
29          }
30      }
31  }
```

Клас PlayingState, що реалізує IstreamState

```
1   using System;
2   namespace OnlineRadioStation.Domain
3   {
        2 references
4       public class PausedState : IStreamState
5       {
            5 references
6           private readonly DjStream _stream;
            1 reference
7           public PausedState(DjStream stream)
8           {
9               _stream = stream;
10          }
11
            2 references
12          public void Play()
13          {
14              Console.WriteLine("Cannot play a paused stream. Use Resume.");
15          }
            2 references
16          public void Stop()
17          {
18              Console.WriteLine("Stream is stopping from pause...");
19              _stream.SetState(new StoppedState(_stream));
20          }
            2 references
21          public void Pause()
22          {
23              Console.WriteLine("Stream is already paused.");
24          }
            2 references
25          public void Resume()
26          {
27              Console.WriteLine("Stream is resuming...");
28              _stream.SetState(new PlayingState(_stream));
29          }
30      }
31  }
```

Клас PausedState, що реалізує IstreamState

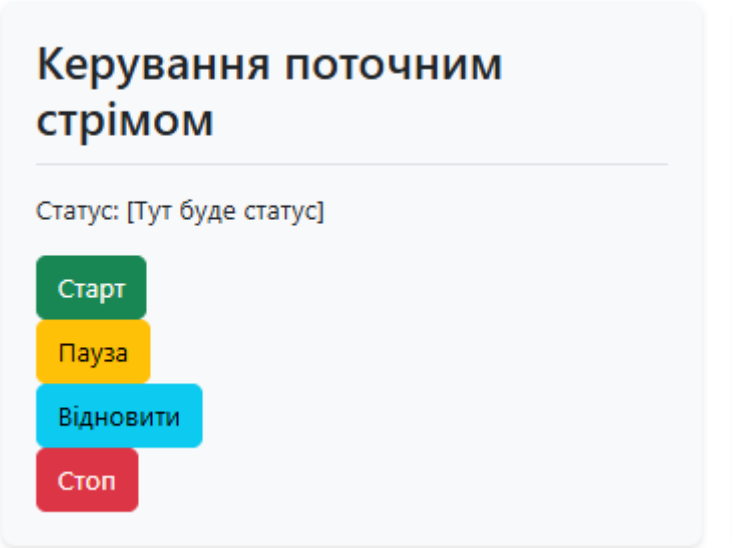


Рисунок 2 – Реалізовано зміна статусу стріму

5) Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

3. Контрольні питання:

1. Що таке шаблон проєктування?

Шаблон проєктування — це універсальне рішення для типових проблем у розробці ПЗ, яке описує структуру та взаємодію об’єктів.

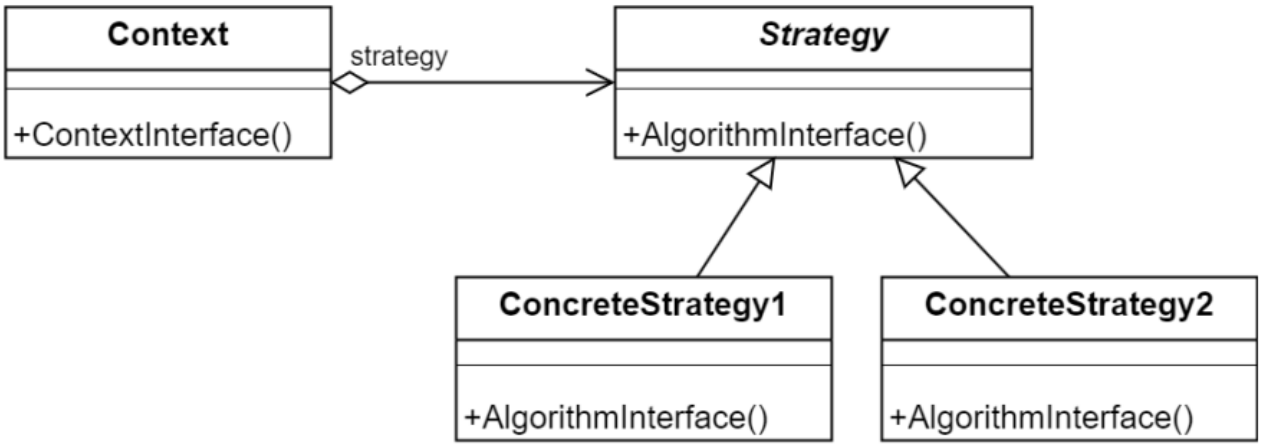
2. Навіщо використовувати шаблони проєктування?

- Спрощують розробку.
- Покращують читабельність і масштабування коду.
- Допомагають уникнути помилок.
- Забезпечують гнучкість і повторне використання.

3. Яке призначення шаблону «Стратегія»?

Шаблон Стратегія дозволяє динамічно вибирати алгоритми, інкапсулюючи їх у взаємозамінні класи, щоб уникнути умовних конструкцій.

4. Нарисуйте структуру шаблону «Стратегія».



5. Які класи входять в шаблон «Стратегія», та яка між ними взаємодія?

Класи:

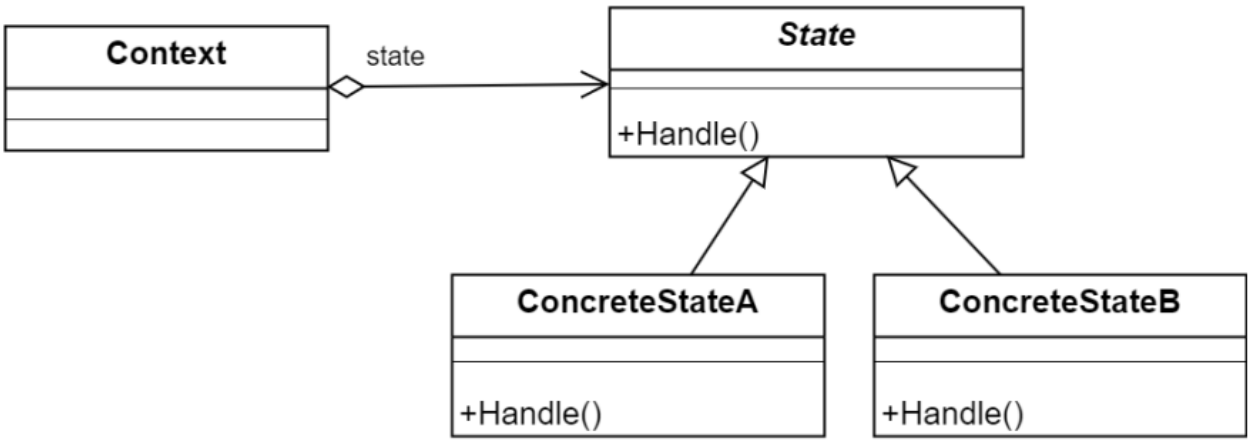
- Strategy: Інтерфейс із методом algorithm().
- ConcreteStrategy: Реалізації алгоритмів.
- Context: Використовує Strategy через setStrategy() і викликає algorithm().

Взаємодія: Контекст делегує виконання алгоритму об’єкту Strategy, який клієнт встановлює динамічно.

6. Яке призначення шаблону «Стан»?

Шаблон «Стан» дозволяє об’єкту змінювати поведінку залежно від стану, інкапсулюючи стани в окремі класи.

7. Нарисуйте структуру шаблону «Стан».



8. Які класи входять в шаблон «Стан», та яка між ними взаємодія?

Класи:

- State: Інтерфейс із методом handle().

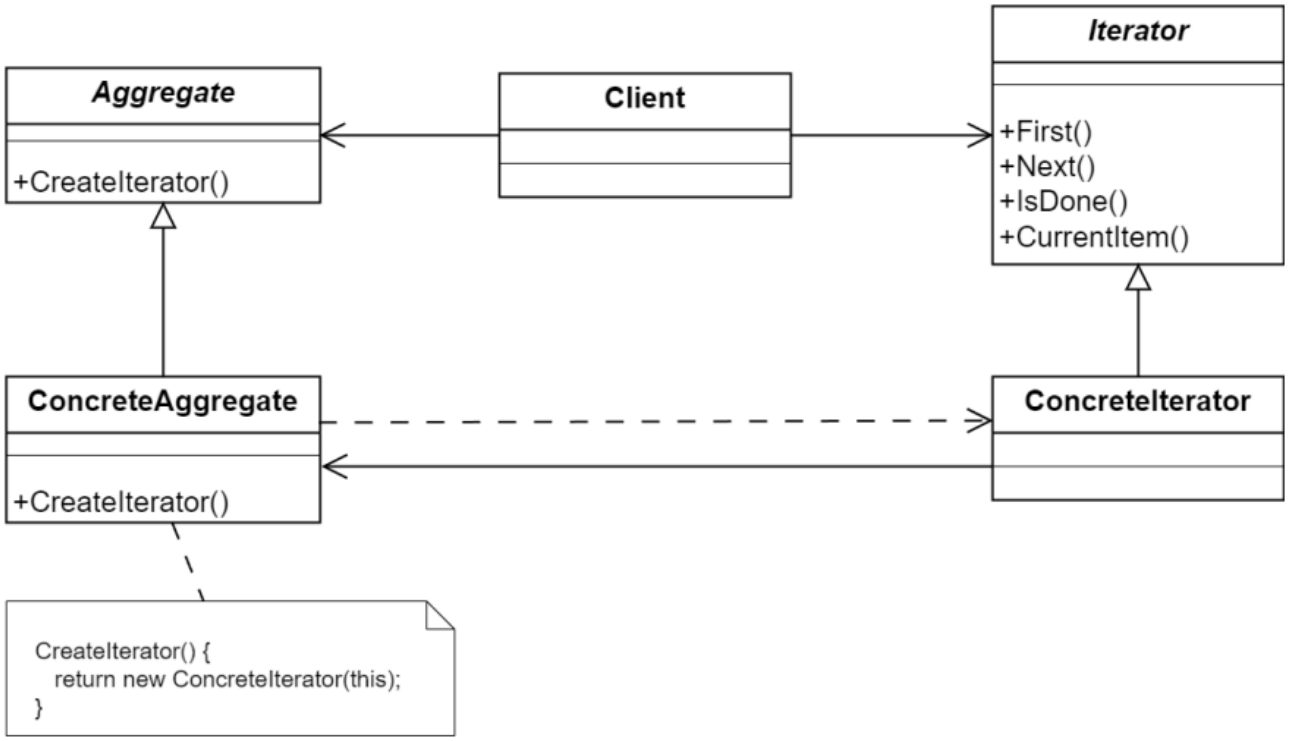
- ConcreteState: Реалізації поведінки для стану.
- Context: Зберігає стан, делегує виконання handle().

Взаємодія: Контекст викликає handle() поточного стану, який може змінити стан через setState().

9. Яке призначення шаблону «Ітератор»?

Шаблон «Ітератор» забезпечує послідовний доступ до елементів колекції, приховуючи її внутрішню структуру.

10. Нарисуйте структуру шаблону «Ітератор».



11. Які класи входять в шаблон «Ітератор», та яка між ними взаємодія?

Класи:

- Iterator: Інтерфейс із методами next(), hasNext().
- ConcreteIterator: Реалізація обходу.
- Aggregate: Інтерфейс із createIterator().
- ConcreteAggregate: Створює ConcreteIterator.

Взаємодія: Клієнт отримує ітератор через createIterator() і використовує його для обходу колекції.

12. В чому полягає ідея шаблона «Одинак»?

Шаблон «Одинак» забезпечує єдиний екземпляр класу з глобальним доступом до нього.

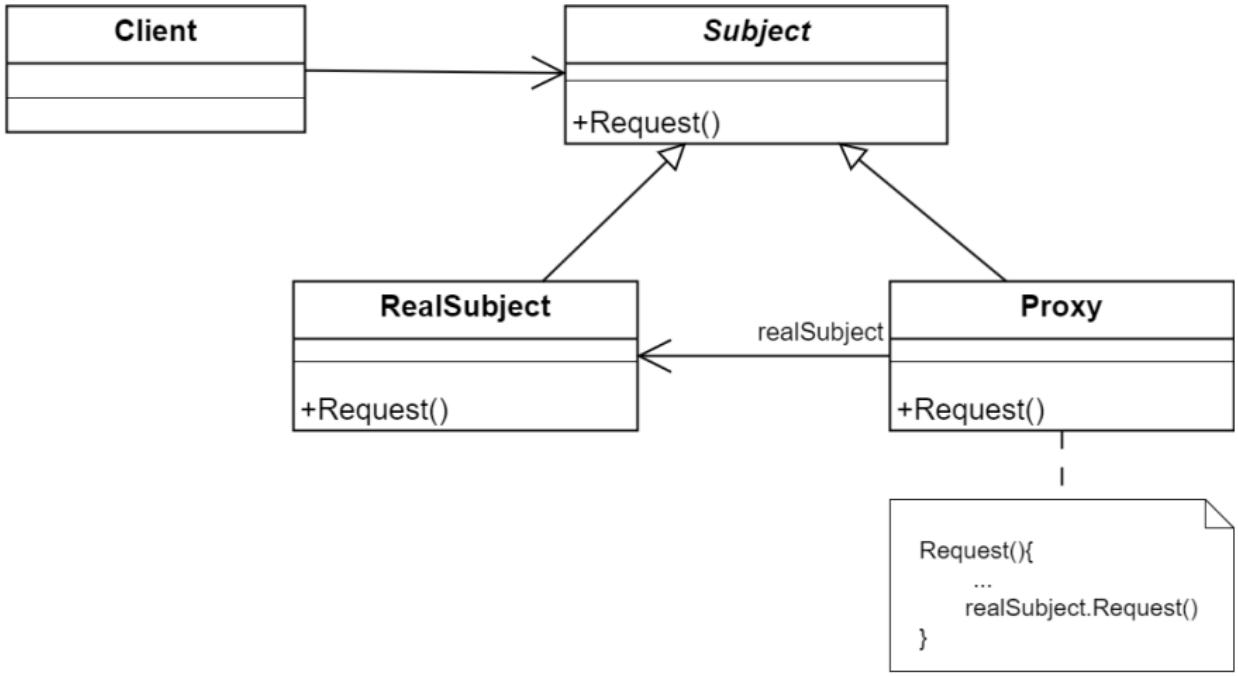
13. Чому шаблон «Одинак» вважають «анти-шаблоном»?

Бо він порушує принципи ООП: ускладнює тестування, створює приховані залежності та глобальний стан

14. Яке призначення шаблону «Проксі»?

Шаблон «Проксі» контролює доступ до об’єкта, додаючи функціонал, як-от ледарська ініціалізація чи перевірка прав.

15. Нарисуйте структуру шаблону «Проксі».



16. Які класи входять в шаблон «Проксі», та яка між ними взаємодія?

Класи:

- Subject: Інтерфейс із методом request().
- RealSubject: Виконує основну роботу.
- Proxy: Контролює доступ до RealSubject.

Взаємодія: Клієнт викликає request() через Proxy, який делегує виклик до RealSubject або додає логіку.

4. Висновок

У ході виконання лабораторної роботи було розглянуто базові шаблони проєктування, зокрема Singleton, Iterator, Proxy, State та Strategy. Отримані знання дали можливість зрозуміти, яку роль відіграють патерни у побудові гнучких і масштабованих програмних систем. На прикладі веб-застосунку «Online Radio Station» було реалізовано шаблон State для організації керування станами об'єкта DjStream (наприклад, Зупинено, Відтворюється, На паузі). Це дало змогу винести логіку, специфічну для кожного стану, в окремі класи та спростити керування переходами між станами стріму. Такий підхід робить клас DjStream чистішим та дозволяє легко додавати нові стани в майбутньому без значних змін у вже написаному коді.